

2D Arrays

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

1. Motivation for 2D Arrays
2. Declaration and Initialization
3. Input and Output
4. Basic Matrix Operations
5. Matrix Multiplication
6. Row and Column Operations
7. Matrix Transpose
8. Higher-Dimensional Arrays

Motivation for 2D Arrays



The Problem: Storing Multi-Dimensional Data

Scenario: A class of 5 students, each enrolled in 3 subjects.

Store the marks of all students in all three subjects: CSE101, ENG101, EEE101.

Using 1D Arrays:

```
int s1[3] = {78, 83, 82};  
int s2[3] = {90, 88, 94};  
int s3[3] = {71, 73, 78};  
int s4[3] = {97, 96, 95};  
int s5[3] = {89, 93, 90};
```

What if we have 120 students in the class?

We would need 120 separate array declarations!

The Solution: 2D Arrays

Store this information in a two-dimensional array:

- **First dimension:** Which student (0, 1, 2, 3, or 4)
- **Second dimension:** Which subject (0, 1, or 2)

		[0]	[1]	[2]
		CSE	ENG	EEE
[0]	S0	78	83	82
[1]	S1	90	88	94
[2]	S2	71	73	78
[3]	S3	97	96	95
[4]	S4	89	93	90

`marks[5][3]`

Declaration and Initialization



2D Array Declaration

Syntax

```
datatype array_name[row_size][column_size];
```

Example: `int matrix[3][4];`

	Col 0	Col 1	Col 2	Col 3
Row 0				
Row 1				
Row 2				

- matrix has $3 \times 4 = 12$ integer elements
- Total memory: $12 \times 4 = 48$ bytes (assuming 4-byte integers)

2D Array with Values

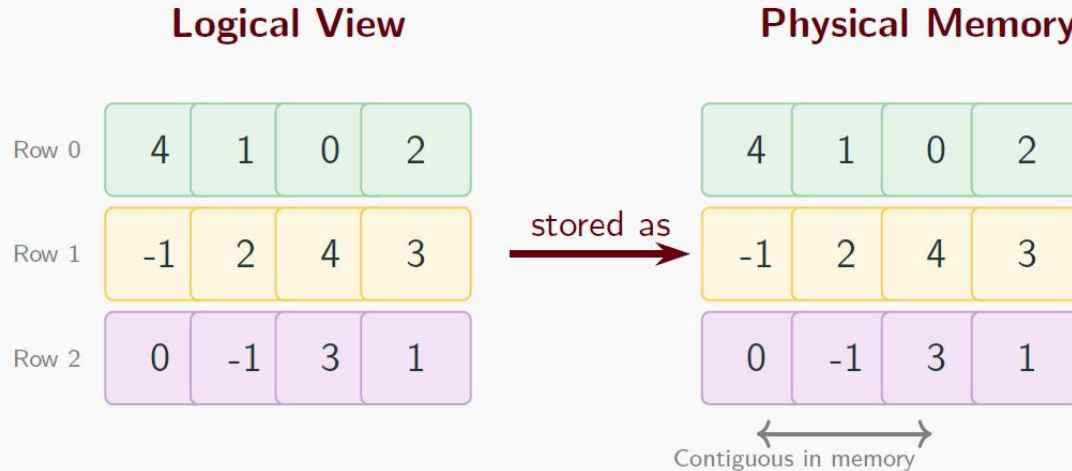
```
int matrix[3][4];
```

	Col 0	Col 1	Col 2	Col 3
Row 0	4	1	0	2
Row 1	-1	2	4	3
Row 2	0	-1	3	1

- `matrix[0][0] = 4` (first row, first column)
- `matrix[1][2] = 4` (second row, third column)
- `matrix[2][3] = 1` (last row, last column)

Memory Layout of 2D Arrays

Key Insight: 2D arrays are stored in row-major order in memory.



Elements are stored **row by row** in consecutive memory locations.

Initialization Method 1: Individual Assignment

```
int m[3][4];
```

```
m[0][0] = 4; m[0][1] = 1; m[0][2] = 0; m[0][3] = 2;  
m[1][0] = -1; m[1][1] = 2; m[1][2] = 4; m[1][3] = 3;  
m[2][0] = 0; m[2][1] = -1; m[2][2] = 3; m[2][3] = 1;
```

4	1	0	2
-1	2	4	3
0	-1	3	1

Initialization Method 2: Single Braces

```
int m [3][4] = {4, 1, 0, 2, -1, 2, 4, 3, 0, -1, 3, 1};
```

Values are assigned **row by row** automatically.

4	1	0	2
-1	2	4	3
0	-1	3	1

Initialization Method 3: Nested Braces (Recommended)

```
int m[3][4] = {  
    {4, 1, 0, 2},  
    {-1, 2, 4, 3},  
    {0, -1, 3, 1}  
};
```

This is the **clearest and most readable** way to initialize 2D arrays.

4	1	0	2
-1	2	4	3
0	-1	3	1

Partial Initialization

```
int m[3][4] = {  
    {4, 1, 5},           // Only 3 values for row 0  
    {-1, 2, 4, 3},      // All 4 values for row 1  
                        // Row 2 not specified  
};
```

Omitted values are automatically set to **0**.

Col 0	Col 1	Col 2	Col 3
4	1	5	0
-1	2	4	3
0	0	0	0

Yellow = auto-filled with 0

Omitting Row Size

When initializing at declaration, you can **omit the row size**.

```
int m[][4] = {  
    {4, 1, 0, 2},  
    {-1, 2, 4, 3},  
    {0, -1, 3, 1}  
}; // Compiler deduces 3 rows
```

Important

You can omit the first dimension (rows), but you must always specify the second dimension (columns)!

Why? The compiler needs to know how many elements are in each row to calculate memory addresses correctly.

Input and Output



Reading 2D Array: Fixed Size

```
#include <stdio.h>

int main () {
    int i, j, matrix[3][4];

    printf("Enter 12 elements:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            scanf ("%d", &matrix[i][j]);
        }
    }

    return 0;
}
```

Nested loops: Outer loop iterates over rows, inner loop iterates over columns.

Reading 2D Array: Variable Size

```
#include <stdio.h>

int main() {
    int i, j, m, n;

    printf("Enter rows and columns: ");
    scanf("%d%d", &m, &n);

    int matrix[m][n]; // Variable length array

    printf("Enter %d elements:\n", m * n);
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf ("%d", &matrix[i][j]);
        }
    }

    return 0;
}
```

Displaying 2D Array

```
#include <stdio.h>
int main() {
    int m [4][3] = {
        {0, 1, 2}, {1, 2, 3}, {2, 3, 4}, {3, 4, 5}
    };
    for (int i = 0; i < 4; i ++) {
        for (int j = 0; j < 3; j ++) {
            printf ("%d ", m[i][j]);
        }
        printf ("\n"); // Newline after each row
    }
    return 0;
}
```

Output:

```
0 1 2
1 2 3
2 3 4
3 4 5
```

Initialization Using Loops

```
int i, j, matrix[3][4];  
  
// Initialize with row index  
for (i = 0; i < 3; i++)  
    for (j = 0; j < 4; j++)  
        matrix[i][j] = i; // Each row has same values
```

`matrix[i][j] = i;`

0	0	0	0
1	1	1	1
2	2	2	2

`matrix[i][j] = j;`

0	1	2	3
0	1	2	3
0	1	2	3

Exercise: Initialize Pattern

Task: Write nested loops to create this pattern:

j:	0	1	2
i=0	0	1	2
i=1	1	2	3
i=2	2	3	4
i=3	3	4	5

Solution:

```
int x [4][3];
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 3; j ++)
        x[i][j] = i + j;
```

Basic Matrix Operations



Finding Maximum Element

Task: Find the maximum value in a 2D array.

```
int matrix[3][4] = {
    {0, 5, 2, 3},
    {-1, 2, 4, 3},
    {0, -1, 3, 1}
};

int max = matrix[0][0];
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        if ( matrix[i][j] > max ) {
            max = matrix[i][j];
        }
    }
}
printf("Maximum = %d\n", max); // Output: 5
```

Counting Occurrences

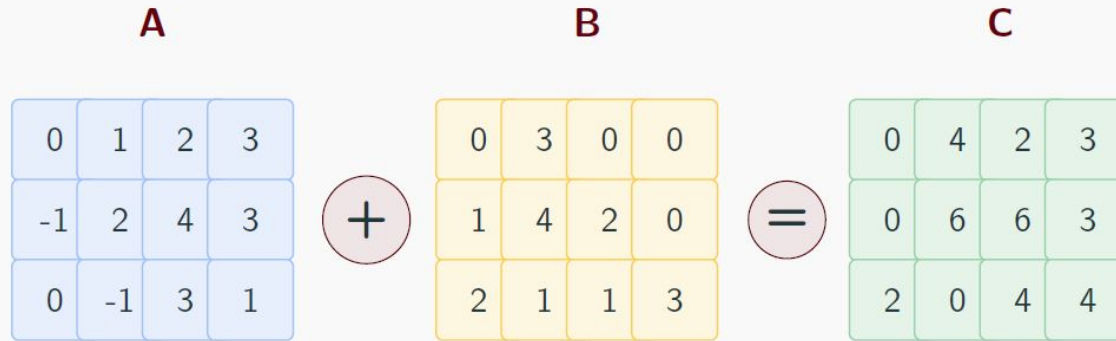
Task: Count how many times a value x appears in the matrix.

```
int matrix[3][4] = {
    {0, 0, 2, 3},
    {-1, 2, 4, 3},
    {0, -1, 3, 1}
};
int x = 0;

int count = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        if (matrix[i][j] == x) {
            count++;
        }
    }
}
printf("Count of %d = %d\n", x, count); // Output : 3
```

Matrix Addition

Rule: Add corresponding elements: $C[i][j] = A[i][j] + B[i][j]$



Both matrices must have the same dimensions!

Matrix Addition: Code

```
int matrix1[3][4], matrix2[3][4], sum[3][4];  
  
// ... initialize matrix1 and matrix2 ...  
  
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 4; j++) {  
        sum[i][j] = matrix1[i][j] + matrix2[i][j];  
    }  
}
```

Matrix Subtraction works the same way

```
diff[i][j] = matrix1[i][j] - matrix2[i][j];
```

Matrix Multiplication



Matrix Multiplication: The Logic

To calculate the result matrix C from $A \times B$:

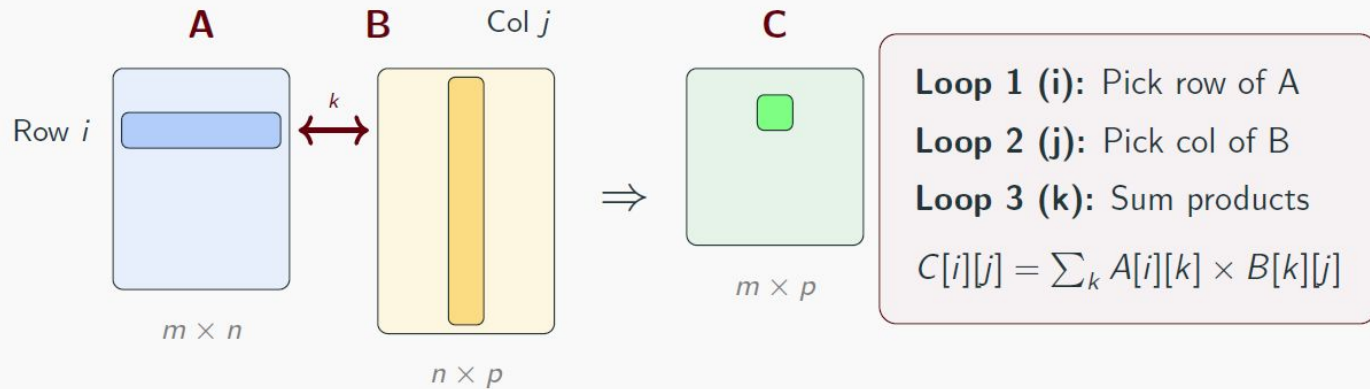
- We need to calculate each element $C[i][j]$ individually
- $C[i][j]$ = **dot product** of Row i from A and Column j from B



Columns in A must equal Rows in B !

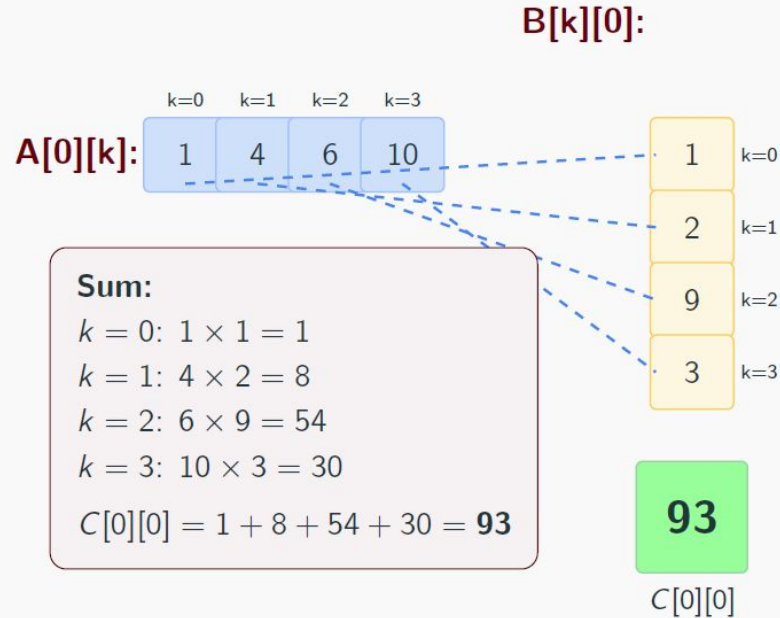
Understanding the Three Nested Loops

Goal: Fill each cell $C[i][j]$ using a sum over index k



Step-by-Step: Calculating $C[0][0]$

Target: $C[0][0]$ = Row 0 of A · Column 0 of B



Matrix Multiplication: Code

```
// A is m x n, B is n x p, C will be m x p
int A[2][4], B[4][3], C[2][3];
int m = 2, n = 4, p = 3;

// ... initialize A and B ...

// Initialize C to zeros
for (int i = 0; i < m; i++)
    for (int j = 0; j < p; j++)
        C[i][j] = 0;

// Matrix multiplication
for (int i = 0; i < m; i++) {
    for (int j = 0; j < p; j++) {
        for (int k = 0; k < n; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Row and Column Operations



Exchange Two Rows

Swap rows i and j :



Exchange Two Rows: Code

```
void swapRows(int a[][3], int i, int j, int cols) {
    for (int k = 0; k < cols; k++) {
        int temp = a[i][k];
        a[i][k] = a[j][k];
        a[j][k] = temp;
    }
}

int main() {
    int a[4][3] = {
        {4, 6, 2},
        {0, 5, 3},
        {0, 8, 1},
        {2, 1, 4}
    };

    swapRows(a, 0, 3, 3); // Swap row 0 and row 3

    return 0;
}
```

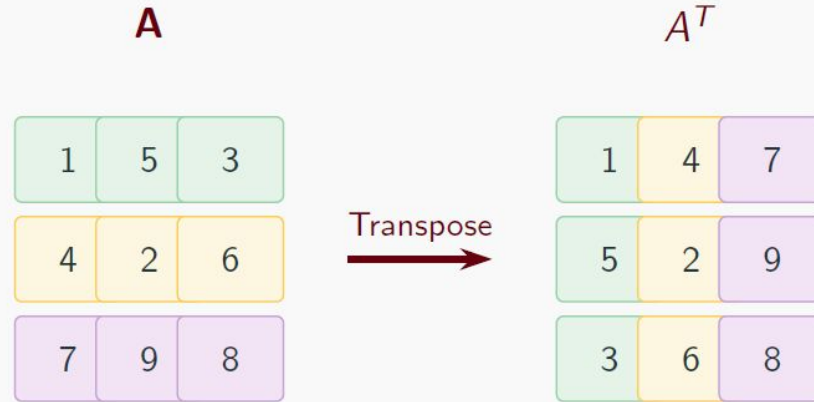
Matrix Transpose



Transpose: Concept

Transpose: Rows become columns, columns become rows.

For element at (i, j) , its transposed position is (j, i) .



- Original: $3 \times 3 \rightarrow$ Transpose: 3×3
- If original is $m \times n$, transpose is $n \times m$

Transpose Using a Separate Array

```
int N = 3;
int a[N][N], b[N][N];

// ... initialize a ...

// Copy a to b with transposed indices
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        b[j][i] = a[i][j];
    }
}
```

Key: $b[j][i] = a[i][j]$ swaps row and column indices.

In-Place Transpose (Square Matrices Only)

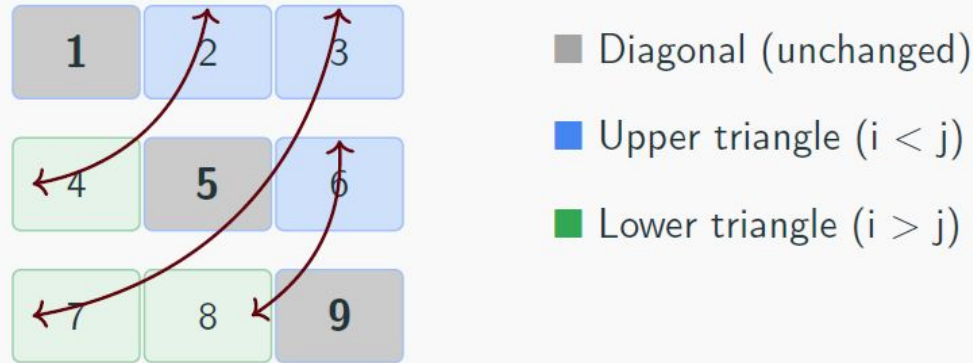
For square matrices, we can transpose without using extra array.

```
int N = 3;
int a[N][N];
// ... initialize a ...
// Swap elements across the diagonal
for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) { // Only upper triangle
        int temp = a[i][j];
        a[i][j] = a[j][i];
        a[j][i] = temp;
    }
}
```

Why $j = i + 1$?

- Diagonal elements ($i = j$) stay in place
- Only swap each pair once (avoid swapping back)

In-Place Transpose: Visualization



We only iterate over the upper triangle (blue) and swap with the corresponding element in the lower triangle (green).

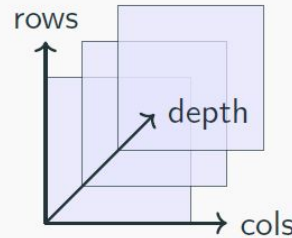
Higher-Dimensional Arrays



3D Arrays and Beyond

C supports arrays with any number of dimensions.

```
double coord[100][100][100]; // 3D array
```



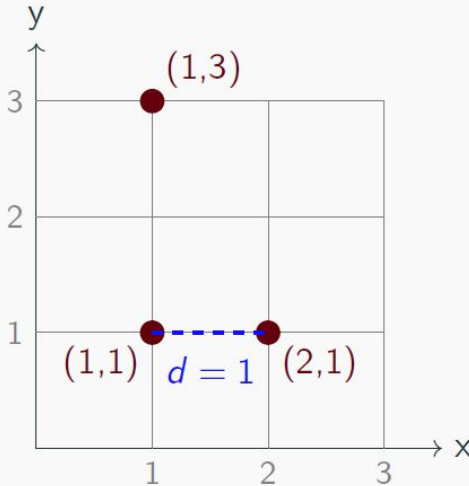
array[depth][rows][cols]

Applications:

- 3D coordinates in space
- Multiple 2D matrices (e.g., RGB image = 3 color channels)
- Time series of matrices

Example: Distance Between Points

Problem: Store points in 2D space and find the closest pair.



Storage: `double p[3][2]` where `p[i][0]` is x and `p[i][1]` is y.

Closest Pair: Code

```
double p[3][2] = {{1, 1}, {2, 1}, {1, 3}};  
double dx, dy, d1, d2, d3, min;  
  
// Distance between point 0 and point 1  
dx = p[0][0] - p[1][0];  
dy = p[0][1] - p[1][1];  
d1 = dx * dx + dy * dy; // Squared distance  
  
// Distance between point 0 and point 2  
dx = p[0][0] - p[2][0];  
dy = p[0][1] - p[2][1];  
d2 = dx * dx + dy * dy;  
  
// Distance between point 1 and point 2  
dx = p[1][0] - p[2][0];  
dy = p[1][1] - p[2][1];  
d3 = dx * dx + dy * dy;  
  
min = (d1 < d2) ? d1 : d2;  
min = ( min < d3) ? min : d3;  
// min = 1 (squared distance between (1,1) and (2,1))
```

Summary



Summary

Key Concepts:

- 2D arrays store data in rows and columns
- Declared as `type name[rows][cols]`
- Stored in row-major order in memory
- Accessed using `arr[i][j]`

Initialization:

- Nested braces (recommended)
- Single brace list
- Nested loops

Common Operations:

- Matrix addition/subtraction
- Matrix multiplication
- Finding max/min
- Row/column swapping
- Transpose

Remember:

- Always specify column size
- Use nested loops for traversal
- Be careful with bounds!

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET